

FOR DATA SCIENTISTS AND ANALYSTS

Data Cleaning with Pandas

Tips, tricks and patterns to turn messy real-world data into clean, analysis-ready DataFrames - fast.

WHAT THIS GUIDE TEACHES YOU TO DO

BEFORE - Raw messy data

	name	age	salary
0	Alice	NaN	50,000
1	ALICE	28	52000
2	Bob	999	NaN
3	bob	31	61000



AFTER - Clean ready data

	name	age	salary
0	Alice	28.0	50000
1	Bob	31.0	61000

10

CHAPTERS

28

PAGES

50+

CODE EXAMPLES

20+

VISUAL DEMOS

What's Inside

From raw and messy to clean and analysis-ready. Every technique with visual before/after examples.

THE BASICS

01	Why Data Cleaning Matters	3
02	First Look - Exploring Your DataFrame	5
03	Handling Missing Values (NaN)	7

DATA TYPES AND STRUCTURE

04	Data Types and Type Casting	9
05	Strings and Text Cleaning	11
06	Duplicates and Index Resetting	13

ADVANCED TECHNIQUES

07	Outliers - Detection and Handling	15
08	Reshaping and Merging DataFrames	17
09	Common Mistakes to Avoid	19
10	Pro Tips and Tricks	21

REFERENCE

--	Summary and Key Takeaways	24
--	Cheat Sheet	26
--	Thank You	28



How to use this guide

Each chapter shows the concept, the code, and a visual before/after side by side. Run the examples yourself - muscle memory beats reading. The cheat sheet on page 26 is your daily reference.

Why Data Cleaning Matters

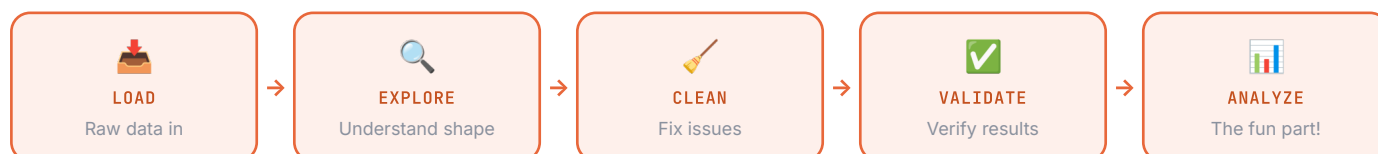
Garbage in, garbage out. 80% of a data scientist's time is spent cleaning data. This chapter explains why - and how to make peace with it.



EVERY DATA SCIENCE MOVIE VS REALITY

Movies: Train the neural network. **Reality:** Why is age stored as 'twenty-eight' in column 3?

The Data Cleaning Pipeline



What dirty data costs you

Wrong insights, failed models, business decisions based on fiction. A model trained on dirty data confidently gives wrong answers.



What clean data gives you

Reliable analysis, accurate models, and stakeholders who trust your outputs. Clean data is the foundation of every good data product.

THE GOLDEN RULE

Never clean data in-place without a backup. Always keep a copy of the raw data. Use `df = df_raw.copy()` before you start cleaning - make this a reflex.

Common Types of Dirty Data

PROBLEM TYPE	EXAMPLE	RISK	FIX
Missing values	<code>NaN</code> , <code>None</code> , empty string	High	<code>fillna</code> / <code>dropna</code>
Wrong data types	age as <code>"28"</code> string	Medium	<code>astype</code> / <code>to_numeric</code>
Duplicates	Same row twice after joins	Medium	<code>drop_duplicates</code>
Inconsistent text	"NYC", "new york", "N.Y.C"	High	<code>str.lower</code> / <code>replace</code>
Outliers	age = 999, salary = -1	High	IQR / clip
Wrong format	"2024/01/15" vs "15-01-2024"	Medium	<code>to_datetime</code>
Extra whitespace	" Alice "	Low	<code>str.strip</code>

safe_start.py

PYTHON

```
import pandas as pd
import numpy as np

# Always start with a safe copy!
df_raw = pd.read_csv("data.csv")
df = df_raw.copy()      # never touch df_raw again

# The 5-line health check - run on every new dataset
print(df.shape)         # (rows, cols) - understand the scale
print(df.dtypes)        # column data types - spot type bugs
print(df.isnull().sum()) # missing count per column
print(df.duplicated().sum()) # duplicate rows total
print(df.describe())     # stats - spot outliers fast
```



The 5-line health check - run on every new dataset

shape tells you scale, dtypes reveals type bugs, `isnull().sum()` shows missing data, `duplicated()` catches join issues, `describe()` surfaces outliers. Five commands, full picture. 30 seconds.

First Look – Exploring Your DataFrame

Before you clean anything, understand what you are dealing with. These are the commands you run in the first 5 minutes of seeing any new dataset.

Essential Exploration Commands

explore.py

PYTHON

```
# Quick overview
df.head(5)           # first 5 rows
df.tail(5)           # last 5 rows (catches trailing junk!)
df.sample(10)        # 10 random rows - avoids head bias
df.info()            # dtypes + non-null counts
df.describe()         # stats for numeric columns
df.describe(include='object') # stats for text cols

# Column-level investigation
df['city'].value_counts()           # frequency table
df['city'].value_counts(normalize=True) # as %
df['city'].nunique()                # number of unique values
```

Reading df.info() – Annotated

Learn to read this in 5 seconds - it tells you almost everything

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
#   Column   Non-Null  Dtype
---  ---
0   name     998      object  ← 2 missing!
1   age      850      float64 ← 150 missing!
2   city     1000     object
3   salary   720      float64 ← 280 missing!
4   joined   1000     object  ← should be datetime!
```

Measuring Missing Data

missing_check.py

PYTHON

```
# % missing per column - one liner
missing_pct = (df.isnull().sum() / len(df) * 100).sort_values(ascending=False)
print(missing_pct[missing_pct > 0])

# Custom percentiles - spot outliers better
df.describe(percentiles=[.01, .05, .25, .75, .95, .99])

# Find columns with only 1 unique value (useless columns!)
useless = [col for col in df.columns if df[col].nunique() == 1]
print("Drop these:", useless)

# Column names with any null values
null_cols = df.columns[df.isnull().any()].tolist()
```

OUTPUT - % missing per column

salary	28.0%
age	15.0%
name	0.2%
city	0.0%
joined	0.0%

PRO TIP - YDATA-PROFILING

Install `ydata-profiling` and run `ProfileReport(df)` for a full automated HTML report covering distributions, correlations, and missing data in one click. Perfect for the first look at any new dataset.

COLUMNS TO ALWAYS CHECK

1) Non-null count - missing data. 2) dtype - wrong types. 3) value_counts on categoricals - unexpected values. 4) min/max on numerics - outliers. 5) nunique on identifiers - duplicates.

Handling Missing Values (NaN)

NaN is not your enemy - it is information. The question is never just how do I remove NaN? but why is it missing, and what should I do about it?

The Three Types of Missing Data



MCAR - Missing Completely at Random

Missing for no reason related to the data. Safe to drop or impute with mean/median. Example: sensor hiccup lost 3 readings.



MAR - Missing at Random

Missing depends on other observed columns. Impute using related columns. Example: income missing more for younger people.



MNAR - Not at Random

Missing depends on the missing value itself. Danger zone! High earners skip the salary field. Dropping will bias your data.

Visual: missing pattern in data - red cells = NaN

col_A	col_B	col_C	col_D	col_E	col_F
42	yes	NaN	3.1	A	100
17	NaN	hi	NaN	B	200
NaN	no	bye	1.7	NaN	150
88	yes	NaN	2.9	A	NaN
55	NaN	ok	4.2	C	300

☐ Valid data ☐ Missing (NaN)

dropna vs fillna - When to Use Which

handle_nulls.py

PYTHON

```
# dropna - use when rows with NaN are truly unusable
df.dropna() # drop rows with ANY NaN
df.dropna(subset=['salary']) # drop only where salary is NaN
df.dropna(thresh=4) # keep rows with at least 4 non-NaN

# fillna - use when you want to preserve rows
df['age'].fillna(df['age'].mean()) # mean (symmetric data)
df['age'].fillna(df['age'].median()) # median (skewed - safer)
df['city'].fillna('Unknown') # placeholder for strings
df['price'].fillna(method='ffill') # forward fill - time series!

# Fill all numeric columns with their median at once
num = df.select_dtypes(include='number').columns
df[num] = df[num].fillna(df[num].median())

# Group-aware fill - smarter imputation!
df['salary'] = df['salary'].fillna(
    df.groupby('city')['salary'].transform('median')
)
```

BEFORE FILLNA(MEDIAN)

	name	age
0	Alice	NaN
1	Bob	28
2	Carol	34
3	Dan	NaN

AFTER FILLNA(MEDIAN=31.0)

	name	age
0	Alice	31.0
1	Bob	28.0
2	Carol	34.0
3	Dan	31.0

WATCH OUT - INPLACE VS REASSIGN

fillna() and dropna() do NOT modify in place by default. You must reassign: `df['col'] = df['col'].fillna(0)` or use `inplace=True`. Forgetting this is the most common pandas bug.

Data Types and Type Casting

A column that looks like numbers but is stored as strings will silently break every calculation. Type errors are sneaky, widespread, and completely preventable.

Pandas Data Types - Visual Guide

Common dtypes and when to use them

int64	INTEGER	Age, count, ID - no decimals, no NaN allowed
float64	FLOAT	Salary, price, ratio - decimals and NaN ok
object	STRING	Names, cities, free text fields
bool	BOOLEAN	True/False flags, active/inactive columns
datetime64	DATETIME	Timestamps - enables time arithmetic
category	CATEGORY	Low-cardinality strings - 10x memory savings!

type_casting.py

PYTHON

```
# Fix common type problems
df['age'] = pd.to_numeric(df['age'], errors='coerce') # safe! NaN on fail
df['joined'] = pd.to_datetime(df['joined'])          # parse dates
df['city'] = df['city'].astype('category')           # save memory!

# Remove thousands separator before converting
df['salary'] = df['salary'].str.replace(',', '').astype('float')
```

to_numeric errors='coerce' - Before and After

BEFORE - MIXED COLUMN (OBJECT)

	score	dtype
0	85	object
1	N/A	object
2	92	object
3	error	object
4	77	object

AFTER - TO_NUMERIC(COERCE)

	score	dtype
0	85.0	float64
1	NaN	float64
2	92.0	float64
3	NaN	float64
4	77.0	float64

memory_trick.py

PYTHON

```
# to_numeric with errors='coerce' converts bad values to NaN
df['score'] = pd.to_numeric(df['score'], errors='coerce')

# Memory optimization - convert low-cardinality strings to category
for col in df.select_dtypes(include='object').columns:
    n_unique = df[col].nunique()
    n_total = len(df)
    if n_unique / n_total < 0.5: # less than 50% unique = good candidate
        df[col] = df[col].astype('category')
```

OUTPUT - Memory savings from category dtype

```
city (object):    800,000 bytes
city (category):  80,000 bytes ← 10x smaller!
```

Strings and Text Cleaning

Real-world string data is a disaster zone. 'New York', 'new york', 'NEW YORK', ' New York ' are all the same city. Pandas does not know that. You have to tell it.

The .str Accessor – Your Text Cleaning Toolkit

string_cleaning.py

PYTHON

```
# The holy trinity of string normalization
df['city'] = df['city'].str.strip().str.lower() # chain them!

# Replace and fix specific patterns
df['phone'] = df['phone'].str.replace(r'^0-9', '', regex=True)
df['email'] = df['email'].str.lower().str.strip()

# Extract parts of strings
df['first_name'] = df['full_name'].str.split().str[0]
df['domain'] = df['email'].str.split('@').str[1]

# Replace known bad values with a mapping
df['gender'].replace({'M': 'Male', 'F': 'Female', 'm': 'Male'}, inplace=True)

# Replace sentinel values with NaN globally
df.replace([999, -1, '', 'N/A', 'na', 'n/a'], np.nan, inplace=True)
```

BEFORE - INCONSISTENT CITIES

	city
0	new york
1	NEW YORK
2	New York
3	n.y.

AFTER - STRIP().LOWER()

	city
0	new york
1	new york
2	new york
3	n.y.

Regex-Based Extraction and Validation

regex_extract.py

PYTHON

```
# Extract structured data from messy strings using regex
# Input: "Age: 28 years, DOB: 1995-04-12"
df['age'] = df['raw'].str.extract(r'Age: (\d+)')
df['dob'] = df['raw'].str.extract(r'DOB: (\d{4}-\d{2}-\d{2})')

# Validate email format - flag bad ones
email_pat = r'^[\w\.-]+@[ \w\.-]+\.\w+$'
df['email_valid'] = df['email'].str.match(email_pat)

# Filter rows where text CONTAINS a pattern
df[df['notes'].str.contains('urgent', na=False, case=False)]

# Normalize all column names in one line
df.columns = df.columns.str.lower().str.replace(' ', '_').str.strip()
```

STR.CONTAINS FOR FILTERING

Use `df[df['col'].str.contains('pattern', na=False)]` to filter rows matching a pattern. The `na=False` prevents NaN from crashing the filter - always include it.

NORMALIZE COLUMN NAMES IMMEDIATELY

Column names like 'First Name ', 'AGE', 'Salary (USD)' are painful to work with. Run the column rename one-liner right after loading any CSV. It prevents bugs for the entire session.

abc

The string cleaning order that always works

1) strip() to remove whitespace. 2) lower() to normalize case. 3) replace() to fix known variants. 4) regex to extract or validate. 5) map() to standardize categories. In that order, every time.

Duplicates and Index Resetting

Duplicates inflate counts, skew averages, and corrupt group aggregations. Finding them is easy. Deciding which copy to keep is the interesting part.

Finding and Removing Duplicates

duplicates.py

PYTHON

```
# Find duplicates
df.duplicated().sum()          # how many?
df[df.duplicated(keep=False)]  # show ALL copies

# Remove duplicates
df.drop_duplicates(inplace=True)      # keep first
df.drop_duplicates(keep='last')       # keep most recent
df.drop_duplicates(subset=['name','email']) # by specific cols

# After dropping, ALWAYS reset the index!
df = df.reset_index(drop=True) # drop=True avoids old index column
```

BEFORE DROP_DUPLICATES

	name	email
0	Alice	a@x.com
1	Alice	a@x.com
2	Bob	b@x.com
3	Bob	b@x.com

AFTER + RESET_INDEX

	name	email
0	Alice	a@x.com
1	Bob	b@x.com

DUPLICATES AFTER MERGES

Always check `df.shape` before and after any merge. Unexpected duplicate keys create row explosions: 100 rows x 100 rows on a shared key = 10,000 rows. Use `validate='1:1'` to catch this.

Outliers – Detection and Handling

Outliers are data points that don't fit the pattern. Sometimes they're errors (age=999). Sometimes they're gold (a viral post with 10M views). Learn to tell the difference.



IQR Method (Robust)

Outlier if below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$. Works for skewed data. Not affected by the outliers themselves. The classic box plot rule.



Z-Score Method (Normal)

Outlier if $|z\text{-score}| > 3$. Assumes roughly normal distribution. Affected by extreme values. Better for symmetric bell-curve data.

outliers.py

PYTHON

```
# IQR method - the robust choice (not affected by extremes)
Q1 = df['salary'].quantile(0.25)
Q3 = df['salary'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Option 1: Remove outliers
df_clean = df[(df['salary'] ≥ lower) & (df['salary'] ≤ upper)]

# Option 2: Clip outliers (cap them - preserves row count!)
df['salary'] = df['salary'].clip(lower=lower, upper=upper)

# Flag outliers without removing - useful for ML features
df['is_outlier'] = ~df['salary'].between(lower, upper)
```

CLIP VS DROP

Prefer `.clip()` over dropping when you need to preserve row count (e.g. for ML training). Clipping sets extreme values to the boundary instead of removing the whole row. Much safer for model training sets.

Reshaping and Merging DataFrames

Real data is rarely in one clean table. You'll join, pivot, melt, and concat your way to the shape you need. Here's how to do it without making a mess.

Merge Types - Visual Guide

INNER MERGE EXAMPLE - ONLY ROWS WITH MATCHING ID ARE KEPT

LEFT (df_a)

	id	name
0	1	Alice
1	2	Bob
2	3	Carol

+

RIGHT (df_b)

	id	salary
0	1	50000
1	2	60000
2	4	70000

=

INNER MERGE

	id	name	salary
0	1	Alice	50000
1	2	Bob	60000

merging.py

PYTHON

```
# All merge types
pd.merge(a, b, on='id', how='inner') # only matching rows
pd.merge(a, b, on='id', how='left')  # all of a, NaN where no match
pd.merge(a, b, on='id', how='outer') # all rows, NaN where missing
pd.concat([a, b], ignore_index=True) # stack rows vertically

# Validate the merge - catch unexpected duplicates!
pd.merge(a, b, on='id', validate='1:1') # raises if not 1-to-1
```

Pivot and Melt – Wide vs Long Format

WIDE FORMAT				
	id	Q1	Q2	Q3
0	A1	100	120	95
1	A2	80	85	90

LONG FORMAT (AFTER MELT)			
	id	quarter	value
0	A1	Q1	100
1	A1	Q2	120
2	A1	Q3	95
3	A2	Q1	80

reshape.py

PYTHON

```
# Pivot - long format to wide format
df_wide = df.pivot(index='user_id', columns='metric', values='value')

# Pivot table (handles duplicate keys with aggregation)
df.pivot_table(index='city', values='salary', aggfunc='mean')

# Melt - wide format back to long format
df_long = df_wide.melt(id_vars=['user_id'], var_name='metric', value_name='value')

# Explode - one row per list item (great for JSON columns!)
df['tags'] = df['tags'].str.split(',') # "a,b,c" → ['a','b','c']
df = df.explode('tags').reset_index(drop=True)
```

LONG VS WIDE - WHEN TO USE EACH

Long format (tidy data) is better for analysis with pandas and seaborn. Wide format is more human-readable. Rule: use long for computation, convert to wide only for presentation or specific ML requirements.

Common Mistakes

These pandas mistakes have wasted thousands of hours of data scientist time. Learn them here. Never repeat them in production.

MISTAKE 1 - SETTINGWITHCOPYWARNING

Modifying a slice of a DataFrame modifies a copy, not the original. Always use `.loc[]`:

`df.loc[df['age'] > 18, 'status'] = 'adult'`. This is the #1 source of silent data bugs.

WRONG - SILENT BUG

```
# May not work - operates on copy!
df[df['age'] > 18]['status'] = 'adult'
# SettingWithCopyWarning!
```

CORRECT - USE .LOC ALWAYS

```
# This always works correctly
df.loc[df['age'] > 18, 'status'] = 'adult'
# Safe and explicit
```

MISTAKE 2 - FORGETTING INPLACE OR REASSIGNMENT

Most pandas methods return a new DataFrame. If you don't reassign or use `inplace=True`, your change vanishes. Use `df = df.dropna()` not just `df.dropna()`.

MISTAKE 3 - COMPARING NAN WITH =

`df['col'] == np.nan` always returns False. NaN is not equal to anything. Always use `df['col'].isnull()` or `df['col'].isna()` to check for missing values.

MISTAKE 4 - USING APPLY() WHEN VECTORIZED OPS EXIST

`df['col'].apply(lambda x: x * 2)` is 10-100x slower than `df['col'] * 2`. Always try vectorized operations first. Use `apply()` only for complex logic that truly cannot be vectorized.

SLOW - PYTHON LOOP (SECONDS)

```
# 10s on 100k rows
results = []
for _, row in df.iterrows():
    results.append(row['price'] * 1.1)
df['price_vat'] = results
```

FAST - VECTORIZED (MILLISECONDS)

```
# 0.001s on 100k rows

df['price_vat'] = df['price'] * 1.1
```

MISTAKE 5 - NOT VALIDATING AFTER CLEANING

You cleaned the data and now trust it blindly. Add assertions! `assert df.isnull().sum().sum() == 0` will scream if any NaN survived. Catch problems at the source, not 3 notebooks later.

MISTAKE 6 - IGNORING ROW COUNT AFTER MERGES

A join on a non-unique key creates a Cartesian product. 3 rows x 3 rows on id=1 gives 9 rows. Always print `df.shape` before and after every merge. Use `validate='1:1'` when you expect unique keys.



ME MERGING TWO DATAFRAMES WITHOUT CHECKING FOR DUPLICATE KEYS

Input: 1,000 rows. Output: 14,000,000 rows. That seems right...

Pro Tips and Tricks

The difference between a junior who takes 2 hours to clean data and a senior who takes 20 minutes. These tricks come from years of real data work.

Method Chaining - Fluent Pandas Style

method_chain.py

PYTHON

```
# Method chaining - one readable block of cleaning logic
df_clean = (
    df_raw
    .copy()
    .dropna(subset=['name', 'salary'])
    .drop_duplicates(subset=['email'])
    .assign(
        name = lambda x: x['name'].str.strip().str.title(),
        salary = lambda x: pd.to_numeric(x['salary'], errors='coerce'),
        joined = lambda x: pd.to_datetime(x['joined'])
    )
    .query('salary > 0 and salary < 500000')
    .reset_index(drop=True)
)
```

WHY CHAIN?

Method chaining keeps your cleaning logic in one readable block. Each step is visible in order. No intermediate variables to track. Easier to debug - just comment out one step to see its effect.



The .assign() + lambda combo is your best friend

Use `.assign(col=lambda x: ...)` to add or transform columns inside a chain. The lambda receives the DataFrame at that point in the chain - so earlier .assign() results are available to later ones.

Datetime Tricks for Cleaning

datetime_tips.py

PYTHON

```
# Parse dates with format specified (faster + safer)
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d')

# Extract useful components
df['year']      = df['date'].dt.year
df['month']     = df['date'].dt.month
df['weekday']   = df['date'].dt.day_name()
df['is_weekend'] = df['date'].dt.dayofweek >= 5

# Days since signup
df['days_active'] = (pd.Timestamp.now() - df['joined']).dt.days

# Flag future dates (likely data entry errors!)
df['is_future'] = df['date'] > pd.Timestamp.now()
```

Validation with Assertions

validate.py

PYTHON

```
# Add assertions after cleaning to catch regressions
assert df.isnull().sum().sum() == 0, "Nulls remain!"
assert df.duplicated().sum() == 0, "Duplicates remain!"
assert df['age'].between(0, 120).all(), "Invalid ages!"
assert df['salary'].gt(0).all(), "Negative salaries!"
assert df.shape[0] >= 100, "Too many rows dropped!"
print("All checks passed! Data is clean.")
```

OUTPUT

```
All checks passed! Data is clean.
```

Powerful One-Liners

oneliners.py

PYTHON

```
# Apply same cleaning to multiple columns at once
str_cols = ['name', 'city', 'dept']
df[str_cols] = df[str_cols].apply(lambda x: x.str.strip().str.title())

# Drop columns where > 50% values are missing
df = df.dropna(axis=1, thresh=int(len(df) * 0.5))

# Drop constant columns (only 1 unique value - useless!)
const = [c for c in df.columns if df[c].nunique() == 1]
df.drop(columns=const, inplace=True)

# Balanced sample - n rows per group
df_balanced = df.groupby('class').sample(n=100, random_state=42)

# Convert all *_date columns to datetime at once
date_cols = [c for c in df.columns if 'date' in c.lower()]
df[date_cols] = df[date_cols].apply(pd.to_datetime, errors='coerce')
```

Hidden Gem Functions



pd.cut() and pd.qcut()

Bin continuous values into categories. `cut()` uses fixed ranges. `qcut()` uses quantiles for equal-size bins. Perfect for age groups or salary tiers.



df.pipe()

Apply a custom function inside a chain:
`df.pipe(fix_phone).pipe(normalize_email)`.
Makes pipelines unit-testable and reusable across projects.



df.where() and df.mask()

`where(cond)` keeps values where True, replaces with NaN elsewhere. `mask(cond)` is the inverse. Great for conditional nulling without `.loc`.



df.filter()

Select columns by name pattern:
`df.filter(like='_id')` or
`df.filter(regex='^Q')`. Much cleaner than listing column names manually.

Performance Tips for Large DataFrames

TECHNIQUE	SPEEDUP	WHEN TO USE
Vectorized operations	100x vs loop	Always - avoid iterrows
category dtype	10x memory	Low-cardinality string columns
int32 instead of int64	2x memory	When values fit in 32-bit range
Specify dtype on read_csv	3x load	Large CSVs you read often
query() for filtering	~2x speed	Complex multi-condition filters
eval() for arithmetic	~5x speed	Math on large DataFrames
Chunked reading	Any file size	Files bigger than available RAM

performance.py

PYTHON

```
# Read large CSV with dtypes - 3x faster load
dtypes = {'age': 'int32', 'salary': 'float32', 'city': 'category'}
df = pd.read_csv("big.csv", dtype=dtypes)

# Process files larger than RAM using chunks
chunks = []
for chunk in pd.read_csv("huge.csv", chunksize=10000):
    clean = chunk.dropna().query("salary > 0")
    chunks.append(clean)
df = pd.concat(chunks, ignore_index=True)

# Use eval() for fast arithmetic on large DataFrames
df.eval('profit = revenue - cost', inplace=True) # 5x faster
```

GroupBy + Transform – The Secret Weapon

groupby_tricks.py

PYTHON

```
# Group-aware imputation (smarter than global median)
df['salary'] = df['salary'].fillna(
    df.groupby('city')['salary'].transform('median')
)

# Flag rows above their group average
df['above_avg'] = df['salary'] > df.groupby('city')['salary'].transform('mean')

# Add group size as a column (useful for filtering small groups)
df['group_size'] = df.groupby('city')['city'].transform('count')
df = df[df['group_size'] ≥ 5] # drop groups smaller than 5

# Rank within group (percentile rank per category)
df['salary_rank'] = df.groupby('dept')['salary'].rank(pct=True)
```

WHY TRANSFORM() AND NOT JUST GROUPBY().AGG()

transform() returns a Series with the same index as the original DataFrame - so you can assign it directly back as a new column. agg() collapses to one row per group. Use transform() when you want group-level stats on every original row.



The complete cleaning checklist

1) df.copy(). 2) df.info() + describe(). 3) Handle nulls. 4) Fix dtypes. 5) Clean strings. 6) Remove dupes + reset_index. 7) Handle outliers. 8) Validate with assertions. Print this. Use it every time.

Key Takeaways

Everything distilled to the essentials. Print this. Stick it on your wall.

1 Always start with `df.copy()`

Never mutate raw data. Keep the original clean. Apply all transformations to a copy and you can always go back. Make this a reflex.

2 The 5-line health check is non-negotiable

`shape`, `dtypes`, `isnull().sum()`, `duplicated().sum()`, `describe()`. Run these on every new dataset before doing anything else. 30 seconds, full picture.

3 NaN is information - think before you drop

Understand WHY data is missing. MCAR vs MAR vs MNAR leads to very different strategies. Dropping blindly introduces bias you may not discover until production.

4 Vectorized operations, always

If you find yourself writing a for loop over DataFrame rows, stop. There is almost certainly a vectorized way that is 100x faster. Learn the patterns once, benefit forever.

5 Validate after every pipeline run

Use assertions to confirm your cleaning worked. Loud errors at the source beat silent corruption 3 notebooks later. Invest 5 minutes in assertions once.



Clean data is not the destination. It's the foundation.

The analysis, the model, the insight - none of it works on dirty data. Invest the time upfront. You will not regret it at 11pm debugging wrong results with a confused stakeholder on a call.

Decision Guide - Which Method When

PROBLEM	METHOD	NOTES
Missing numeric	<code>fillna(median)</code>	Median for skewed; mean for symmetric
Missing categorical	<code>fillna("Unknown")</code>	Or <code>mode()</code> if most frequent makes sense
Missing > 60% of column	Drop the column	Not enough signal to impute reliably
Missing time series	<code>ffill / bfill</code>	Propagate last known value forward
Group-based filling	<code>groupby.transform</code>	Fill with group median - smarter impute
Wrong data type	<code>to_numeric / astype</code>	Always use <code>errors='coerce'</code> for safety
Inconsistent strings	<code>str.strip().lower()</code>	Then <code>replace()</code> known bad variants
Duplicate rows	<code>drop_duplicates</code>	Always <code>reset_index(drop=True)</code> after
Statistical outliers	<code>clip(lower, upper)</code>	Capping is safer than dropping for ML
Sentinel values (999,-1)	<code>replace(val, np.nan)</code>	Then handle as NaN in next step
Same op, many columns	<code>df[cols].apply(func)</code>	One operation, all selected columns

THE 20-MINUTE CLEAN RULE

A practiced data scientist can do first-pass cleaning of most datasets in 20 minutes using this guide. The goal is not perfection - it is clean enough to explore. Perfection comes in iteration 2.

Pandas Cleaning Cheat Sheet

Copy-paste ready commands. Keep this page open while you work.

FIRST LOOK

```
df.shape
df.info()
df.describe()
df.isnull().sum()
df.duplicated().sum()
df.sample(5)
```

MISSING VALUES

```
df.dropna()
df.dropna(subset=['col'])
df['col'].fillna(df['col'].median())
df['col'].fillna('Unknown')
df['col'].fillna(method='ffill')
```

TYPE CASTING

```
pd.to_numeric(df['col'], errors='coerce')
pd.to_datetime(df['col'])
df['col'].astype('category')
df['col'].astype('int32')
df['col'].str.replace(',','').astype('float')
```

STRING CLEANING

```
df['col'].str.strip()
df['col'].str.lower()
df['col'].str.title()
df['col'].str.replace(r'^a-z',' ', regex=True)
df['col'].str.extract(r'(\d+)')
```

DUPLICATES

```
df.duplicated().sum()
df.drop_duplicates()
df.drop_duplicates(subset=['col'])
df.drop_duplicates(keep='last')
df.reset_index(drop=True)
```

OUTLIERS (IQR)

```
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
lo, hi = Q1-1.5*IQR, Q3+1.5*IQR
df['col'].clip(lower=lo, upper=hi)
```

RENAME / FILTER COLS

```
df.columns = df.columns.str.lower().str.replace(' ', '_')
df.rename(columns={'old':'new'})
df.filter(like='_id')
df.filter(regex='^Q')
df.drop(columns=['a','b'])
```

VALIDATION

```
assert df.isnull().sum().sum() == 0
assert df.duplicated().sum() == 0
assert df['age'].between(0,120).all()
assert df['salary'].gt(0).all()
assert df.shape[0] > 100
```

REPLACE VALUES

```
df.replace(999, np.nan)
df.replace(['N/A', 'na', ''], np.nan)
df['col'].map({'A': 'Active', 'I': 'Inactive'})
df['col'].replace({'M': 'Male', 'F': 'Female'})
```

MULTI-COLUMN OPS

```
num = df.select_dtypes(include='number').columns
df[num] = df[num].fillna(df[num].median())

strs = ['name', 'city', 'dept']
df[strs] = df[strs].apply(lambda x: x.str.strip())
```

MERGING

```
pd.merge(a, b, on='id', how='inner')
pd.merge(a, b, on='id', how='left')
pd.concat([a, b], ignore_index=True)
pd.merge(a, b, on='id', validate='1:1')
```

RESHAPE

```
df.pivot(index='id', columns='m', values='v')
df.melt(id_vars=['id'], var_name='m', value_name='v')
df.pivot_table(index='city', values='sal', aggfunc='me
df.explode('tags').reset_index(drop=True)
```

METHOD CHAIN

```
df_clean = (
    df.copy()
    .dropna(subset=['name'])
    .drop_duplicates()
    .assign(name=lambda x: x['name'].str.title())
    .query('salary > 0')
    .reset_index(drop=True)
)
```

DATETIME

```
pd.to_datetime(df['date'], format='%Y-%m-%d')
df['date'].dt.year
df['date'].dt.month_name()
df['date'].dt.dayofweek >= 5 # weekend
(pd.Timestamp.now() - df['date']).dt.days
```

GROUPBY + TRANSFORM ONE-LINER

Fill missing values with the group median: `df['col'].fillna(df.groupby('group')['col'].transform('median'))` - much smarter than filling with the global median.

**Bookmark this page. Refer to it daily.**

These commands cover 95% of data cleaning tasks. The other 5% is Googling edge cases - perfectly fine at any level of data science experience.



Thanks for reading!

This guide is part of the free educational resources from the Big Data Specialist community. Share it with colleagues and help them learn faster.



Instagram

@bigdataspecialist



Telegram

@bigdataspecialist



LearnDevs

learndevs.com



learndevs.com - Everything a developer needs

Free courses, free books, job offers, tech news, roadmaps - all in one place.

Copyright 2026 Big Data Specialist Community - All rights reserved
This guide is free for personal use and sharing with attribution.